S

Ejercicio multi-asset

Análisis y propuesta de solución · Strike Product Designer Challenge

---

Octubre de 2026 · Los mockups en Figma acompañan este documento

Resumen

El problema. Los clientes de Strike no quieren testear un asset aislado, sino varios sistemas conectados como un solo trabajo, por ejemplo la web, la app y la API de una tienda online. Hoy la plataforma solo trabaja asset por asset y no existe forma de armar un ejercicio que junte varios. Crearlo con todos sus assets desde el inicio abre el desafío central de diseño: conviven dos niveles de configuración. Algunas decisiones son del ejercicio completo, como las fechas, y otras son de cada asset, como sus credenciales. El usuario tiene que poder ver y cambiar ambas sin perder de vista qué vale para qué.

La idea. Definir una vez, mostrar las excepciones. El usuario carga los valores generales una sola vez en el ejercicio y cada asset los usa. Si un asset necesita algo distinto, lo cambia, y el sistema lo marca como excepción. Así siempre se ve qué es general y qué es particular.

La solución. Se apoya en tres pilares:

1. **La herencia.** Lo general se define una vez y las excepciones se ven y se pueden revertir.
2. **La visibilidad del impacto.** El usuario ve a qué assets no afecta un cambio general antes de guardarlo.
3. **El sistema inteligente como asistente.** El sistema infiere, sugiere y completa para agilizar la configuración, y el usuario decide.

Qué hay en este documento. El problema, cómo lo abordé, lo que aprendí de la industria, el modelo y cómo responde a los tres desafíos del brief. Las pantallas están en los mockups del Figma, el segundo entregable.

1. El problema, desmenuzado

Lo que pasa hoy

La plataforma de Strike trabaja asset por asset: el cliente administra y revisa cada sistema por separado. Pero al validar el producto apareció un patrón que se repite: casi nadie quiere testear un asset aislado. Quieren revisar varios assets relacionados como parte de un mismo trabajo.

El ejemplo más claro es un e-commerce con sitio web, app y una API que conecta a los dos. Si pide una revisión de seguridad, no le sirve testear cada pieza suelta y por separado: un atacante las aprovecharía en conjunto, así que la revisión también tiene que ser conjunta.

A ese trabajo coordinado el brief lo llama **ejercicio**. Hoy no existe forma de armar uno.

Lo que realmente hay que resolver

La salida más rápida sería sumar un selector de assets al flujo actual. Eso resuelve "juntar", pero deja sin resolver lo central. Apenas se juntan varios assets aparecen dos niveles de decisiones que conviven:

- Algunas son **del trabajo completo**, como las fechas en que se ejecuta.
- Otras son **de cada asset**, porque una app, una API y un sitio web se testean con condiciones distintas: sus credenciales, cuánta carga aguantan, si requieren una VPN.

Por eso entendí que la pregunta de diseño no es dónde ubicar los campos en pantalla, sino **cómo se relacionan los dos niveles**: qué se define una sola vez, qué es propio de cada asset, qué pasa cuando un valor general y uno particular se contradicen, y cómo se entiende todo eso sin confundir al usuario.

Los tres desafíos, como preguntas de diseño

Desafío del brief	La pregunta que me hice	Qué pasa si se resuelve mal
Dos niveles que conviven	¿Cómo sabe el usuario, en cada momento, qué está configurando y a qué alcance aplica?	Cambios generales que pisan excepciones sin avisar, o formularios que se repiten
Alcance variable: de 2 a 40 assets	¿Cómo evito que 40 assets signifiquen 40 formularios idénticos sin complicar el caso de 3?	Un flujo que sirve para un extremo y se rompe en el otro
Perfiles diversos	¿Cómo guío a quien no sabe qué es un rate limit sin frenar a quien sí?	Se abruma al novato o se aburre al experto

Cómo definí el éxito

Antes de diseñar me puse cuatro criterios (propuesta, para ajustar): 1. Crear un ejercicio de 20 assets sin repetir configuración. 2. Que el usuario siempre pueda responder "¿esto afecta a todos o solo a este asset?". 3. Que un cambio general nunca pise una excepción sin que se vea. 4. Que una persona no técnica llegue a un ejercicio válido.

Lo que asumí

Mi clasificación de campos sale de razonar el problema y de contrastarlo con la documentación pública de otras plataformas, no de conocer cómo opera Strike por dentro. Por eso dejo explícitos mis supuestos: los assets ya existen en la plataforma, no diseño la verificación de que el cliente es dueño de cada uno y uso textos placeholder.

2. Cómo lo abordé

Mi proceso

Partí por bajar el problema a tierra y recién después pensé en pantallas. En el camino hice cinco cosas, en este orden:

1. **Entender el problema.** Leí el brief y lo reduje a una pregunta: ¿cómo conviven las decisiones del trabajo completo con las de cada asset?

2. **Clasificar los campos.** El brief da dos listas, una para el ejercicio y otra para el asset. Las ordené en tres categorías: del ejercicio, del asset y las que pueden ser de ambos. Para decidir usé una sola pregunta: ¿este dato es igual para todo el ejercicio o cambia de un asset a otro? Si es igual para todos, es del ejercicio. Si cambia en cada asset, es del asset. Y si hay un valor general que cada asset puede cambiar, es de ambos.
3. **Mirar qué hace la industria.** Revisé cómo resuelven este problema plataformas de pentest y de escaneo de seguridad, y cómo se resuelven la edición en lote y la importación de datos en otros productos.
4. **Definir las reglas de la relación entre niveles.** Qué se hereda, qué pasa cuando un asset pisa un valor, cómo se vuelve atrás y qué se le muestra al usuario cuando cambia algo general.
5. **Diseñar el recorrido completo.** Desde crear el ejercicio hasta administrarlo y editarlo, tanto un asset como varios a la vez. En varias decisiones comparé alternativas antes de elegir.

3. Qué aprendí y tomé de la industria

Revisé cómo plataformas de pentest y de escaneo de seguridad resuelven el alta de assets y la configuración compartida. También miré cómo otros productos resuelven la importación y la edición en lote, porque la escala de 2 a 40 assets exige ambas. No busqué copiar a nadie, busqué qué resolvió bien cada uno y dónde quedó el hueco.

Referentes

Referente	Lo que aprendí	Qué hace	Qué tomé
Cobalt	Esquiva el problema: un asset por pentest obliga a repetir la configuración común	Un pentest por asset, reutilizable; el scope se puede heredar al copiar	Asset desacoplado del trabajo, defaults por intención, ciclo de estados
Horizon3	Las buenas cargas masivas muestran qué se va a crear antes de crearlo	Alta guiada o masiva por CSV o texto; plantillas y ejecuciones recurrentes	Alta masiva; el sistema propone y el usuario confirma
Burp Suite	Tiene la herencia más completa, pero hay que ir a un ícono aparte para verla: existe y está escondida	Herencia por carpetas, gana el nivel más específico; vista del delta	Herencia, "solo lo que cambia", valores comunes al importar
Intruder	Inferir el tipo ahorra pasos de configuración	Etiquetas, alta masiva, sincronización con la nube, detección de tipo	Agrupar e inferir el tipo de asset
Detectify	La sugerencia funciona mejor que pedir todo de entrada	Propone candidatos para revisar y confirmar	El sistema propone y el usuario decide

Referente	Lo que aprendí	Qué hace	Qué tomé
Qualys CSAM	Lo incompleto puede convivir sin bloquear	Lo importado llega como "no gestionado" hasta activarse; detecta duplicados	Estado intermedio (borrador) y detección de duplicados
HackerOne	Separar qué se testea de las reglas ordena la configuración	Scope unificado y reutilizable, dentro y fuera de scope	Scope como entidad reutilizable

Ninguno de los referentes muestra el impacto de un cambio general antes de guardarlo.

Tres oportunidades

Oportunidad	Pilar
Que la herencia se vea en lugar de esconderse	1. La herencia
Mostrar el impacto de un cambio antes de guardar	2. La visibilidad del impacto
Un sistema que sugiere en vez de pedir: defaults por intención y por tipo, y un estado intermedio con revisión previa para lo importado	3. El sistema inteligente como asistente

4. La idea central y los tres pilares

Definir una vez, mostrar las excepciones

La idea es la siguiente: los valores generales se cargan una sola vez y todos los assets los usan. Cuando un asset necesita algo distinto, solo ese asset se marca como excepción. Así, con 40 assets el usuario no configura 40 veces: carga lo general una vez y después solo atiende las excepciones.

Esa idea se sostiene en tres pilares. Cada uno responde a una parte del problema.

Pilar	Problema que resuelve	Piezas que lo sostienen (ver pantallas de Figma)
1. La herencia	Que dos niveles convivan sin confundir	Valores globales heredados y marcados con el planeta, columna "Herencia" (Heredado, Modificado) con su filtro, borde del campo pisado, "Volver a valor original"
2. La visibilidad del impacto	Poder cambiar algo general sin romper lo ya personalizado	Resumen en vivo de a cuántos assets no afecta, confirmación breve al guardar, tooltips de estado

Pilar	Problema que resuelve	Piezas que lo sostienen (ver pantallas de Figma)
3. El sistema inteligente como asistente	La escala de 2 a 40 assets y los perfiles distintos	Tipo de asset inferido, profundidad sugerida según el objetivo, defaults por tipo, borradores en lugar de bloqueos, importación con confirmación

5. El modelo

Qué es del ejercicio, qué es del asset y qué es de ambos

Campo	Nivel	Nota
Título, objetivo, fechas y recurrencia, hacker humano	Del ejercicio	Son iguales para todo el trabajo
Profundidad y franjas horarias	De ambos (el asset hereda y puede cambiarlo)	El ejercicio da el valor base
Credenciales, rate limit, VPN	Propios del asset	Cambian de un asset a otro

Las reglas de la herencia

1. **El vínculo es vivo.** Si cambia el valor del ejercicio, los assets que lo heredan cambian con él.
2. **Un asset puede cambiar un valor heredado.** Cuando lo hace, queda marcado como "Modificado"; si no cambió ninguno, como "Heredado".
3. **Repetir el valor no cuenta.** Si el asset termina con el mismo valor que el ejercicio, no se marca.
4. **Siempre se puede volver atrás.** Cada campo cambiado tiene una acción para volver al valor del ejercicio.

Ejercicio y ejecuciones

El ejercicio es el plan y se programa al crearse. Cada vez que corre, es una ejecución: uno de una sola vez tiene una, uno recurrente tiene varias. Los resultados viven en cada ejecución. El ejercicio tiene tres estados: **Borrador** (faltan datos para que corra), **Activo** (corre según lo programado) y **Pausado** (suspendido por el usuario).

Una vez creado el ejercicio con sus assets, el sistema muestra qué valores faltan para completarlo, por ejemplo la credencial de un asset. Esto pasa con los assets nuevos; los que ya existían y tenían sus datos cargados no piden nada. Esos assets quedan como **Pendiente**; la palabra "Borrador" queda reservada para el ejercicio. Mientras falte algo, el ejercicio queda en Borrador, y pasa a Activo solo cuando se completa todo, sin un botón de activar.

6. Cómo responde a los tres desafíos

Cada desafío del brief tiene una respuesta concreta en la propuesta. Los mockups muestran cómo se ve cada una.

Desafío	Cómo lo resuelvo	Pilar
Dos niveles que conviven	Los assets heredan los valores del ejercicio y solo se marcan cuando se apartan de ellos. Antes de guardar un cambio general, se avisa a cuántos assets no afecta	1. La herencia y 2. La visibilidad del impacto
De 2 a 40 assets	Importación con confirmación previa, edición de varios assets a la vez, buscador y filtros, grupos, vista de diagrama y valores sugeridos por tipo, para no completar un formulario por asset	3. El sistema inteligente
Perfiles técnicos y no técnicos	El sistema sugiere la profundidad del pentest a partir del objetivo definido y explica por qué, los estados se explican al pasar el mouse y lo que falta no bloquea: el asset queda pendiente y se completa después	3. El sistema inteligente

Ayuda para quien no es técnico

Al pasar el mouse sobre cada campo aparece un tooltip que explica qué es. Más adelante pienso sumar un corto tutorial al inicio del flujo principal, para que el sistema enseñe cómo usarlo.

7. Qué dejé afuera y qué quedó abierto

Preferí profundizar en lo central, la creación y la edición del ejercicio con sus dos niveles, antes que cubrir todas las variables de la funcionalidad. Estas son las ideas que pensé y no diseñé:

Extensiones que pensé y no diseñé

- **Historial de corridas:** ver qué pasó en cada ejecución del ejercicio.
- **Corrida manual:** ejecutar el ejercicio fuera de la programación, por ejemplo para volver a testear tras corregir hallazgos.
- **Grupos personalizados:** que el usuario organice sus assets a su manera, siempre sin configuración propia.
- **Búnker de credenciales:** buscar y reutilizar credenciales ya cargadas, con valores enmascarados y registro de uso. En el panel de un asset ya se puede elegir una del búnker o guardar las nuevas; la gestión del búnker queda afuera.
- **Múltiples credenciales por asset:** por ejemplo, una por cada rol.
- **Relación entre assets:** indicar cuáles dependen de cuáles.
- **Conectores con la nube:** traer los assets automáticamente en lugar de importarlos.

Anexo: requisitos

Los requisitos que fijé para la solución visual que se ve en los mockups diseñados en Figma, agrupados por tema.

Tema	Requisito
Herencia	El ejercicio es el padre y los assets son los hijos
	Los valores globales se heredan con vínculo vivo
	Un asset puede cambiar un valor global y queda "modificado"
	Un valor igual al del ejercicio no cuenta como modificado
	La columna "Herencia" marca cada asset como "Heredado" o "Modificado", sin cantidad; el detalle se ve en el asset
	"Volver a valor original", con el valor original en un tooltip
	El campo cambiado lleva un borde del color del tag
Estados	Estados del ejercicio: Borrador, Activo y Pausado, con tooltip
	El ejercicio se programa al crearse, sin paso de lanzar
	Pasa de Borrador a Activo cuando todos los assets tengan su información esencial completa, sin botón "Activar"
	Un asset queda marcado como "Pendiente" cuando faltan datos esenciales; "Borrador" es solo del ejercicio
	Última ejecución visible en el detalle
Creación	Modal central de dos pasos: valores del ejercicio y globales (profundidad y franjas horarias), y después los assets
	Objetivo en desplegable, que sugiere la profundidad y lo explica
	Los valores globales se marcan con un ícono de planeta junto al label, con un tooltip que explica su alcance
	La VPN, el rate limit y las credenciales se completan después de crear
	Elegir assets existentes o importar nuevos
	Importar escribiendo, pegando o subiendo un archivo, con el tipo inferido y editable
	Las filas con error se omiten y las válidas se importan
	Los assets nuevos nacen con valores por defecto según su tipo y las credenciales pendientes

Tema	Requisito
Administración	Lista de ejercicios; al hacer clic en una fila (o Enter) se abre el detalle
	Detalle con configuración y assets en vista de lista; acciones según el estado y aviso de lo que falta en Borrador
	Buscador por nombre, tipo o VPN y un menú "Filtrar" por estado (Listo, Pendiente) y herencia (Heredado, Modificado)
	Selección múltiple con acción de editar, y un lápiz por fila para editar un asset
	Vista de diagrama de la anatomía del ejercicio
	Grupos personalizados que solo organizan
Edición	Un solo componente de panel lateral para el ejercicio, un asset o varios
	Resumen en vivo de cuántos assets no se afectan
	Confirmación al guardar con la cantidad y una lista corta de los assets no afectados
	En el panel de un asset, credenciales con dos opciones: "Cargar credenciales" (del búnker o nuevas) o "No requiere login"
	En el panel de un asset, si cambia un valor global, un diálogo muestra el valor anterior y el nuevo y aclara que el resto de los assets no cambia
	Edición en lote con "Múltiples valores"
	En el lote solo se aplican los campos tocados
	Tooltip en cada campo que explica qué es
Sistema inteligente	Tipo de asset inferido, valores por defecto por tipo y profundidad sugerida según el objetivo